

Федеральное государственное автономное образовательное учреждение  
высшего образования «Национальный исследовательский университет  
ИТМО»

*Факультет программной инженерии и компьютерной техники*  
*Направление подготовки: 09.03.04 – Программная инженерия, Системное*  
*и прикладное программное обеспечение*

Дисциплина: Основы программной инженерии

**Лабораторная работа №4**  
**Вариант №44452**

Выполнил:  
Карнажицкий Максим Романович  
Группа: Р3211

Проверил:  
Осипов С. В.

# Содержание

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| 1. Описание задания .....                                                | 2  |
| 2. Программная реализация .....                                          | 3  |
| 2.1. MBeans .....                                                        | 3  |
| 2.2. JConsole .....                                                      | 4  |
| 2.2.1. Показания разработанных MBeans .....                              | 4  |
| 2.2.2. Версия JVM, поставщик и номер ее сборки .....                     | 4  |
| 2.3. VisualVM .....                                                      | 6  |
| 2.3.1. Показатели MBeans .....                                           | 6  |
| 2.3.2. Имя потока, потребляющего наибольший процент времени<br>CPU ..... | 7  |
| 2.4. Устранение проблем в программе .....                                | 9  |
| 3. Вывод .....                                                           | 17 |

# 1. Описание задания

1. Для своей программы из лабораторной работы 3 по дисциплине «Веб-программирование» реализовать:
  - MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если координаты установленной пользователем точки вышли за пределы отображаемой области координатной плоскости, разработанный MBean должен отправлять оповещение об этом событии.
  - MBean, определяющий процентное отношение «промахов» к общему числу кликов пользователя по координатной плоскости.
2. С помощью утилиты JConsole провести мониторинг программы:
  - Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
  - Определить наименование и версию JVM, поставщика виртуальной машины Java и номер её сборки.
3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:
  - Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
  - Определить имя потока, потребляющего наибольший процент времени CPU.
4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе. По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:
  - Описание выявленной проблемы.
  - Описание путей устранения выявленной проблемы.
  - Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.
  - Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

## 2. Программная реализация

### 2.1. MBeans

Исходный код: [https://github.com/xtern0o/opi\\_lab4/tree/main/src/main/java/org/example/management](https://github.com/xtern0o/opi_lab4/tree/main/src/main/java/org/example/management)

MBeans реализованы с помощью стандартного способа `javax.management.*`, находятся в директории `management/`:

```
+ management/
+-- MBeanRegister      # eager-регистрация на этапе запуска
+-- PointsRatio        # реализация MBean с отношением промахов к кликам
+-- PointsRatioMBean   # интерфейс
+-- PointsStats        # реализация MBean с общей статистикой
+-- PointsStatsMBean   # интерфейс
```

**Вся статистика** (клики, попадания, промахи) собираются в одном **PointsStats**. Для того, чтобы считать отношение в **PointsRatio** берется информация из **PointsStats** через DI.

Но на этом удовольствие не заканчивается. Для того, чтобы увидеть бины в JConsole, их нужно зарегистрировать в рантайме. Это делается в классе **MBeanRegister**.

Так как по умолчанию **@ApplicationScoped**-бины создаются **лениво**, то и наши MBeans зарегаются лениво при первом обращении. Если никто не дергает **MBeanRegister**, **@PostConstruct** не сработает.

На помощь пришел паттерн **Observer**, который запускает метод **register** при событии **@Initialized(ApplicationScoped.class)** - при старте CDI сам создает бин.

## 2.2. JConsole

### 2.2.1. Показания разработанных MBeans

Запускаем JConsole и снимаем показания MBeans:

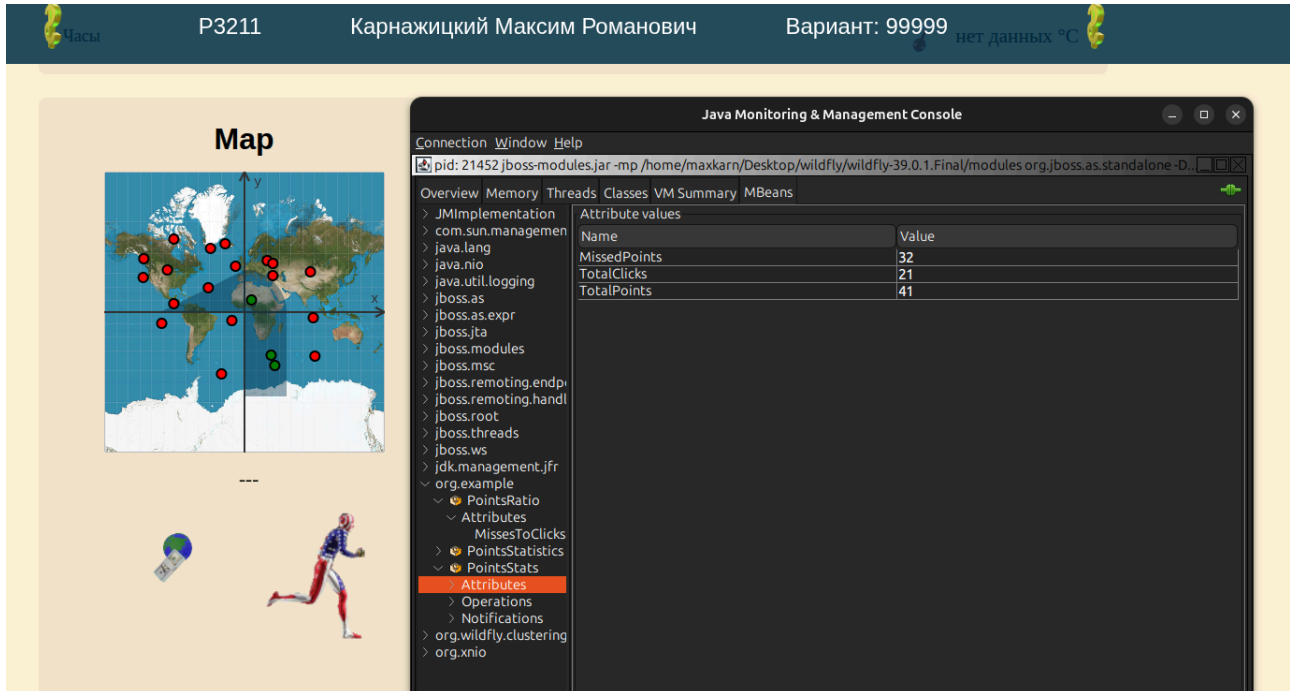


Рис. 1. Показания PointsStatisticsMBean

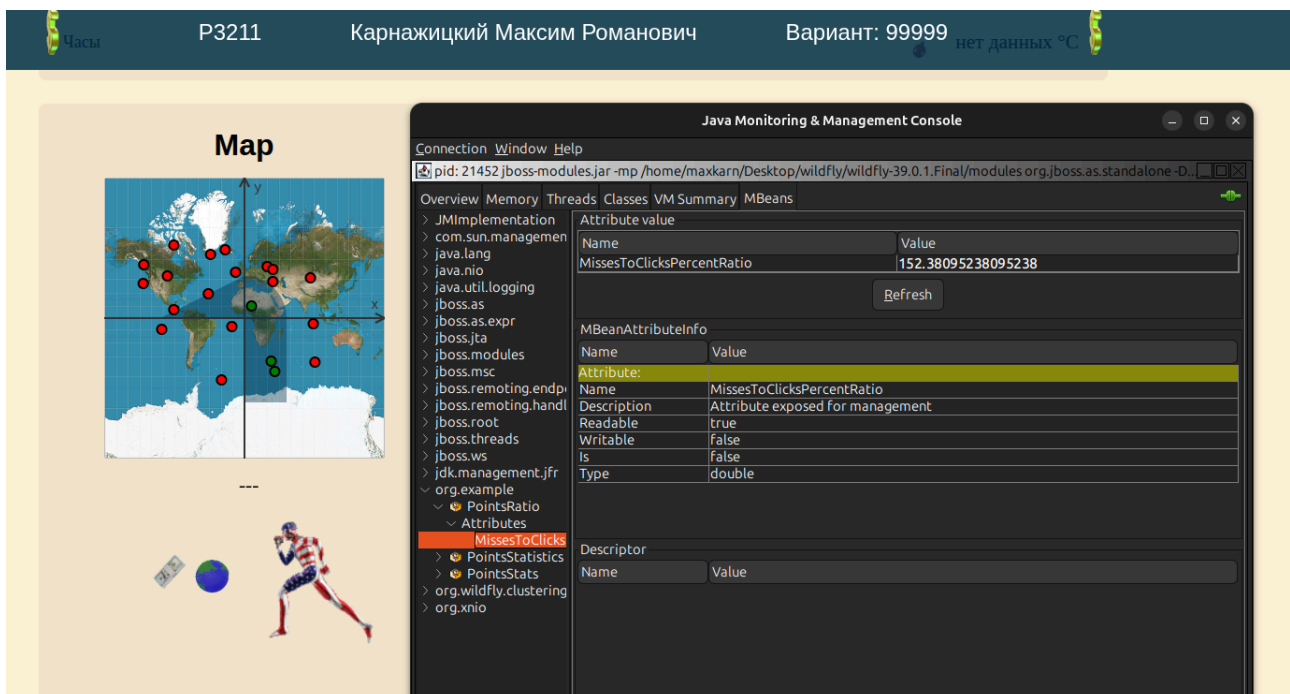


Рис. 2. Показания PointsRatioMBean

### 2.2.2. Версия JVM, поставщик и номер ее сборки

Наименование JVM OpenJDK 64-Bit Server VM

|              |                                               |
|--------------|-----------------------------------------------|
| Версия JVM   | version 21.0.11                               |
| Номер сборки | +10-1-24.04.2-Ubuntu (т. е. <b>build 10</b> ) |
| Поставщик    | Ubuntu                                        |

VM Summary

Monday, June 15, 2026, 11:28:21 PM Moscow Standard Time

Connection name:

pid: 21452 jboss-modules.jar -mp  
/home/maxkarn/Desktop/wildfly/wildfly-39.0.1.Final/modules  
org.jboss.as.standalone -Djboss.home.  
dir=/home/maxkarn/Desktop/wildfly/wildfly-39.0.1.Final -  
Djboss.server.base.dir=/home/maxkarn/Desktop/wildfly/wildfly-  
39.0.1.Final/standalone

Uptime:

13 minutes

Process CPU time:

2 minutes

JIT compiler:

HotSpot 64-Bit Tiered Compilers

Total compile time:

1 minute

Virtual Machine:

OpenJDK 64-Bit Server VM version 21.0.11+10-1-24.04.2-  
Ubuntu

Vendor:

Ubuntu

Name:

21452@maxkarn-BRN-FXXC

Рис. 3. Пруфы что я не вру

## 2.3. VisualVM

### 2.3.1. Показатели MBeans

Снимаем показания MBeans в течение времени, предварительно установив расширение VisualVM-MBeans:

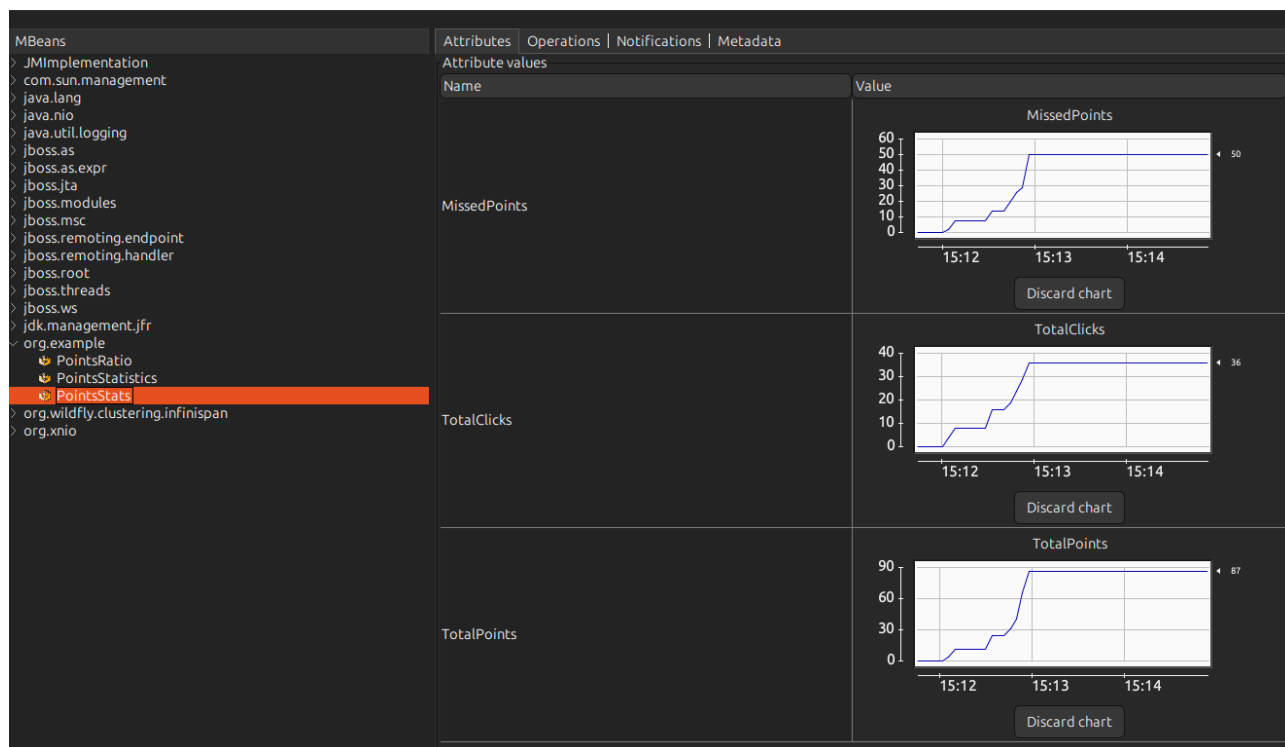


Рис. 4. PointsStats показатели

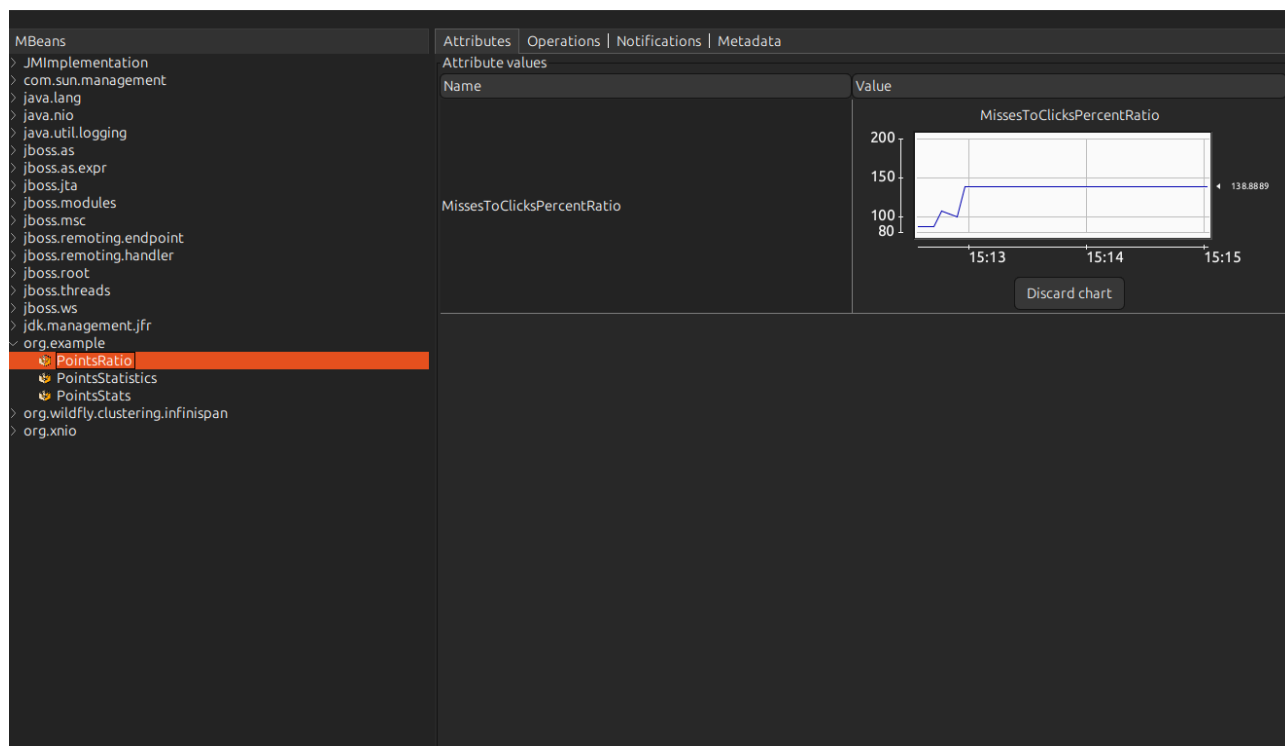


Рис. 5. PointsRatio показатели

Да, интерфейс очень похож на JConsole, поэтому вот пруф, что это правда

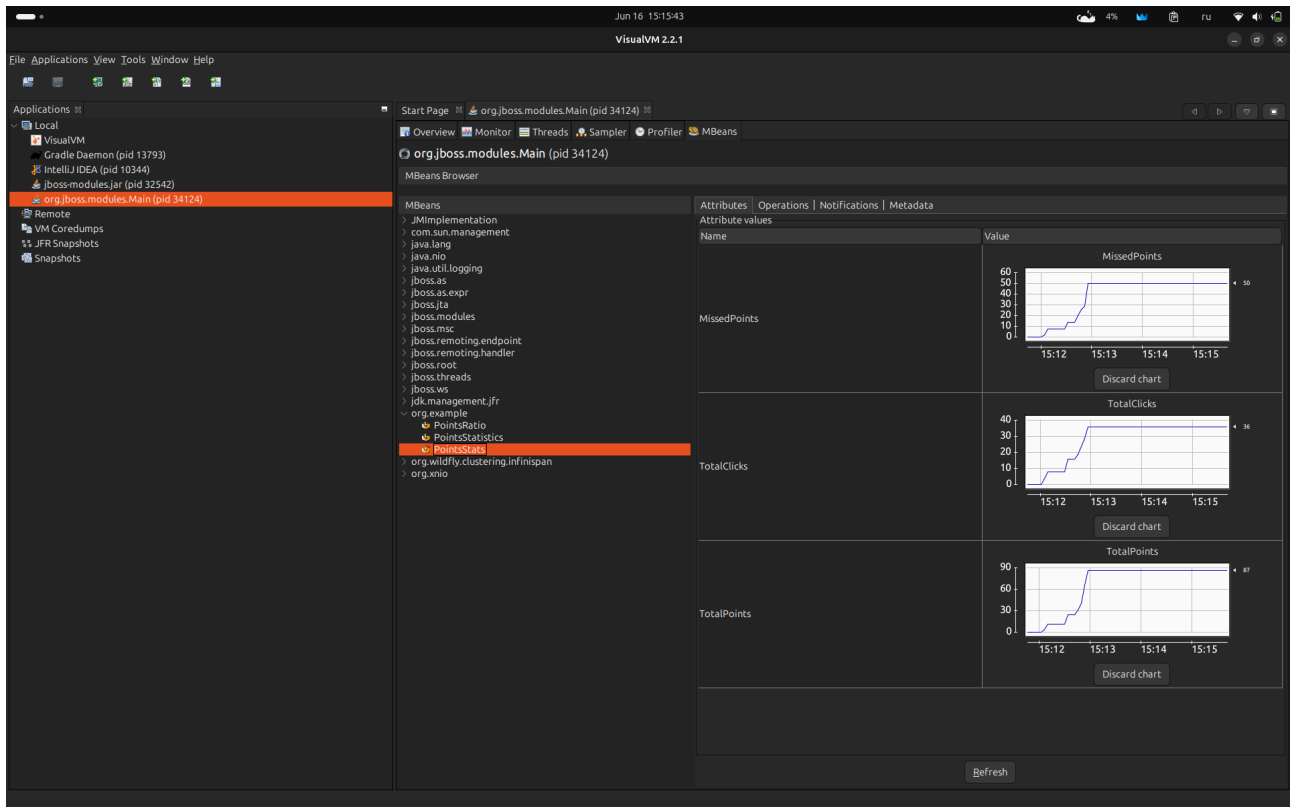


Рис. 6. Это VisualVM

### 2.3.2. Имя потока, потребляющего наибольший процент времени CPU

Чтобы определить поток, потребляющий наибольший процент процессорного времени, заходим в **Sampler** → **CPU** и даём приложению немного поработать: потыкаем точки, полюбуемся фронтендом. Затем переключаемся на вкладку **Thread CPU time** и сортируем по столбцу **Thread Time (CPU)**.

Формально абсолютный максимум приходится на поток **RMI TCP Connection(7)-127.0.0.1: 55.1%** процессорного времени. Однако это служебный поток самого **JMX**-мониторинга: через него VisualVM и снимает данные, поэтому он нагружает CPU именно во время наблюдения (он единственный у кого ненулевой показатель  $/\text{sec} = 6.4\%$ ). К работе приложения он отношения не может иметь.

Среди потоков, отражающих **реальную** работу сервера и приложения, больше всего CPU потребил **default task-2 (9.5%)** - это рабочий поток блокирующего пула веб-сервера **Undertow** (так мне сказали в интернете), на котором WildFly выполняет обработку HTTP-запросов - в момент сэмплинга я создавал новые точки. Потоки вида **MSC service thread 1-N** от-

носятся к контейнеру сервисов WildFly и нагрузили CPU преимущественно на этапе запуска и деплоя, а далее особо не работали.

| Name                            | Thread Time (CPU) | Thread Time (CPU) / sec |
|---------------------------------|-------------------|-------------------------|
| RMI TCP Connection(7)-127.0.0.1 | 36 521 ms (55.1%) | 64.0 ms (6.4%)          |
| RMI TCP Connection(idle)        | 13 294 ms (20.1%) | 0.0 ms (0%)             |
| default task-2                  | 6 315 ms (9.5%)   | 0.0 ms (0%)             |
| MSC service thread 1-3          | 2 225 ms (3.4%)   | 0.0 ms (0%)             |

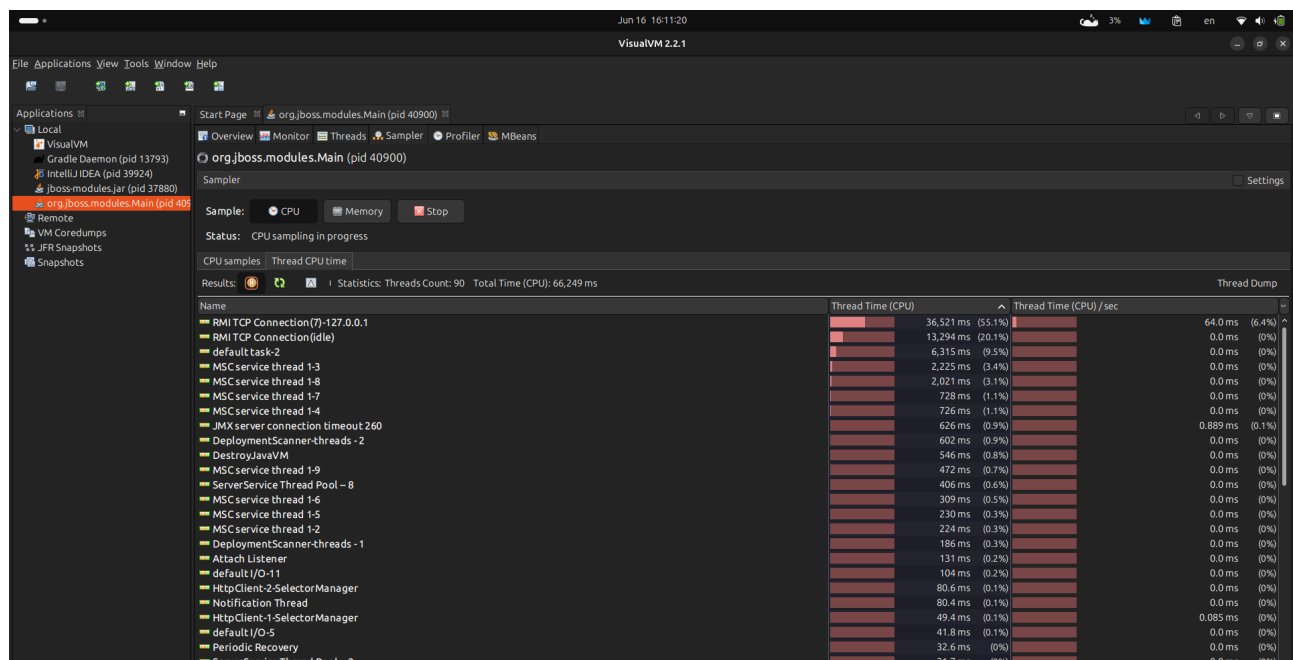


Рис. 7. Скриншот из VisualVM

## 2.4. Устранение проблем в программе

Запускаем наш HttpUnit с фиксированным размером хипа в 30Мб командой:

```
java -Xmx30m -cp "out/production/HttpUnit:lib/*" Main
```

Я просидел 15 минут в ожидании завершения процесса, но он все не завершался. Поэтому я решил удалить строку `Thread.sleep(200)` в классе `Main`, чтобы ускорить процесс. *К тому же это положительно скажется на производительности (думаю, к заданию это не относится)*

Подключаемся к процессу через VisualVM и внимательно смотрим на график HeapSpace.

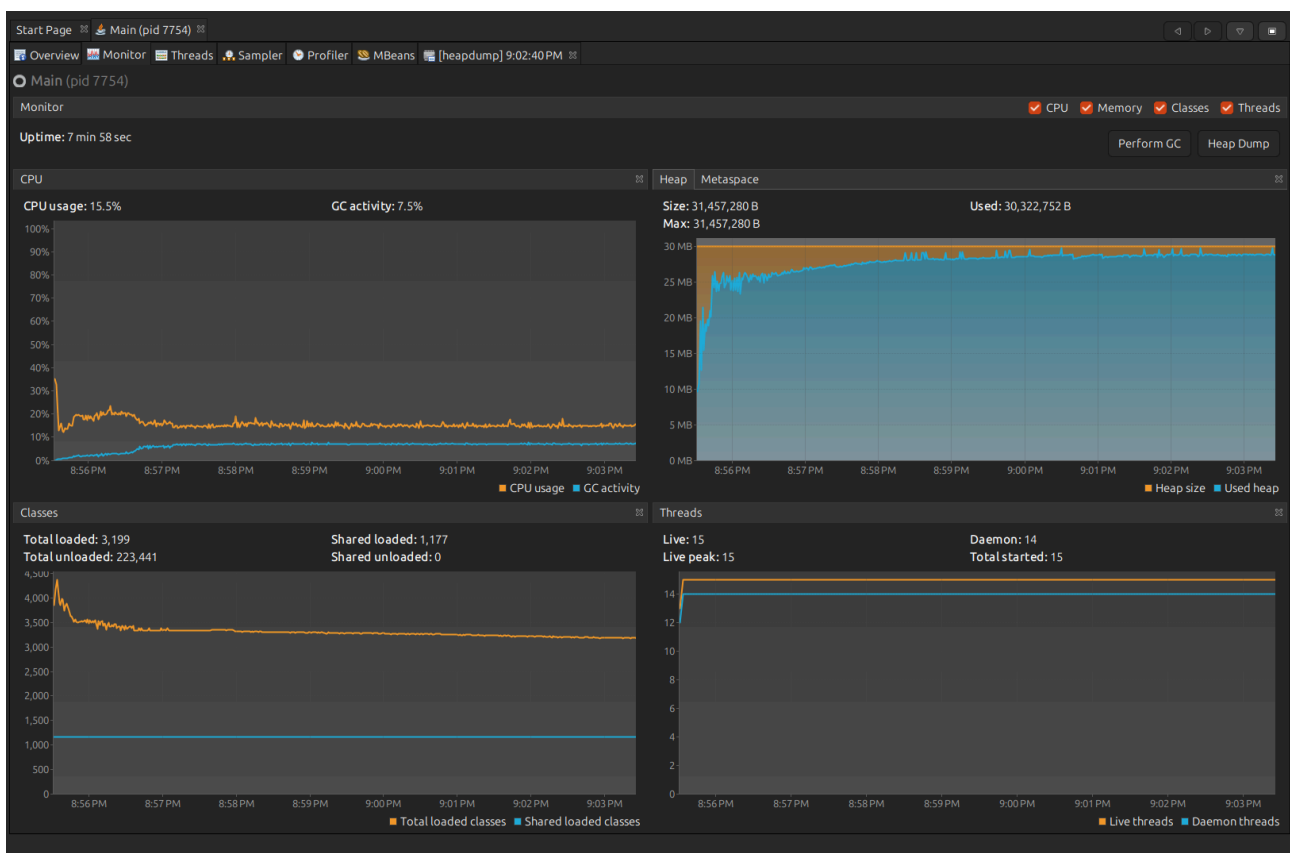


Рис. 8. Данные с вкладки *Monitor* VisualVM

График на самом деле четко дает понять, что это **утечка памяти** - место на хипе постоянно уменьшалось, а GC его не очищал. Программа упала с ошибкой на Count: 96666:

```
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
```

```
Count: 96666[ _response = com.meterware.servletunit.ServletUnitHttpResponse@5cf9db34]
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at java.base/java.lang.StringUTF16.compress(StringUTF16.java:211)
    at java.base/java.lang.String.<init>(String.java:4760)
    at java.base/java.lang.String.<init>(String.java:303)
```

Рис. 9. Ошибка переполнения памяти

Ну что же, пойдём в **heapdump** посмотреть, какие объекты занимают больше всего пространства - снят примерно за 1 минуту до того, как программа упала.

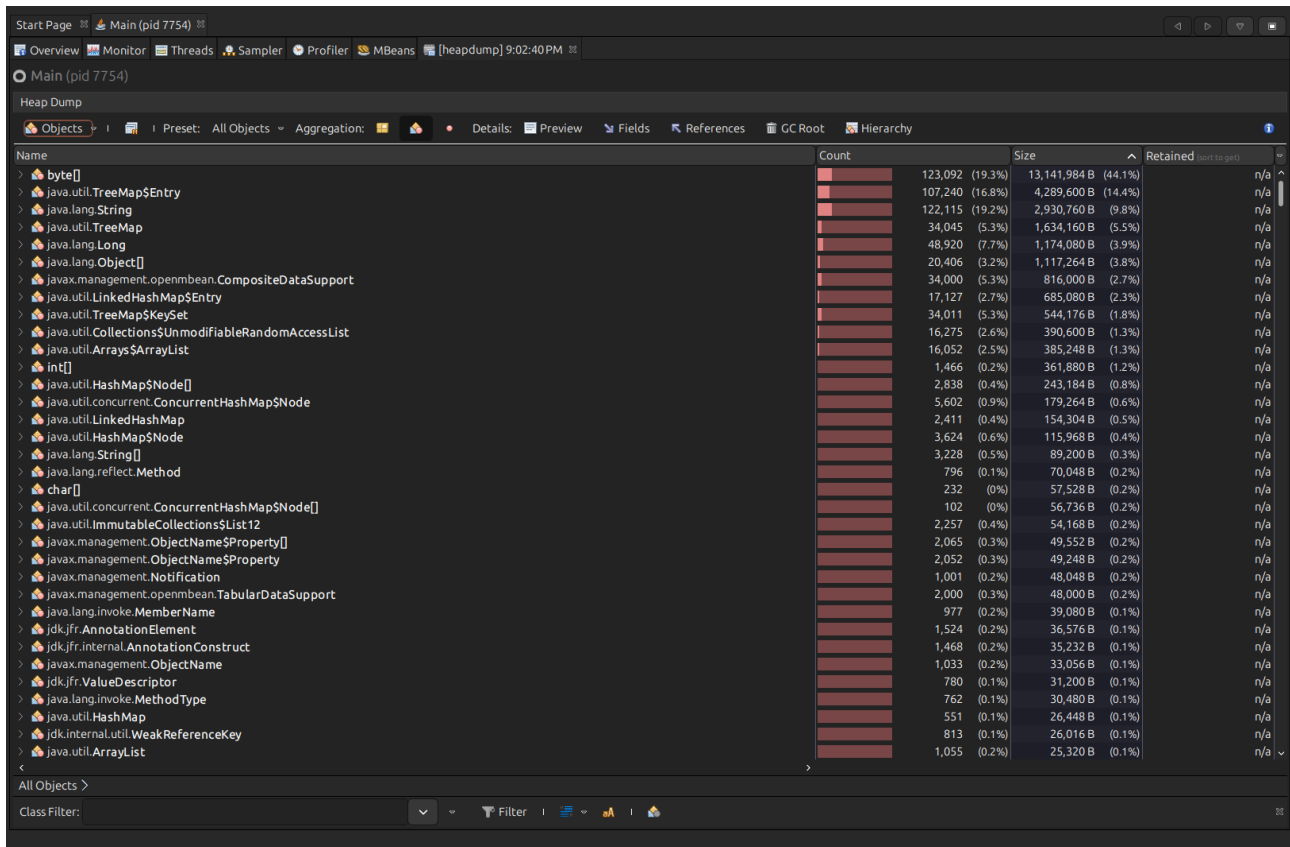


Рис. 10. Состояние кучи примерно за минуту до падения

Из графика работы памяти видно:

- при обработке каждого запроса создаются новые экземпляры **byte[]** и **String** и остаются неочищенными - что-то мне подсказывает, что вряд ли они все далее используются.
- плодятся также некие **TreeMapEntry**

Разберемся с **TreeMapEntry**. Это не классы приложения - это структура JMX. Я прокликал около 100 Entry, и все они относились к **javax.management.\*** или **com.sun.jmx.mbeanserver.\***.

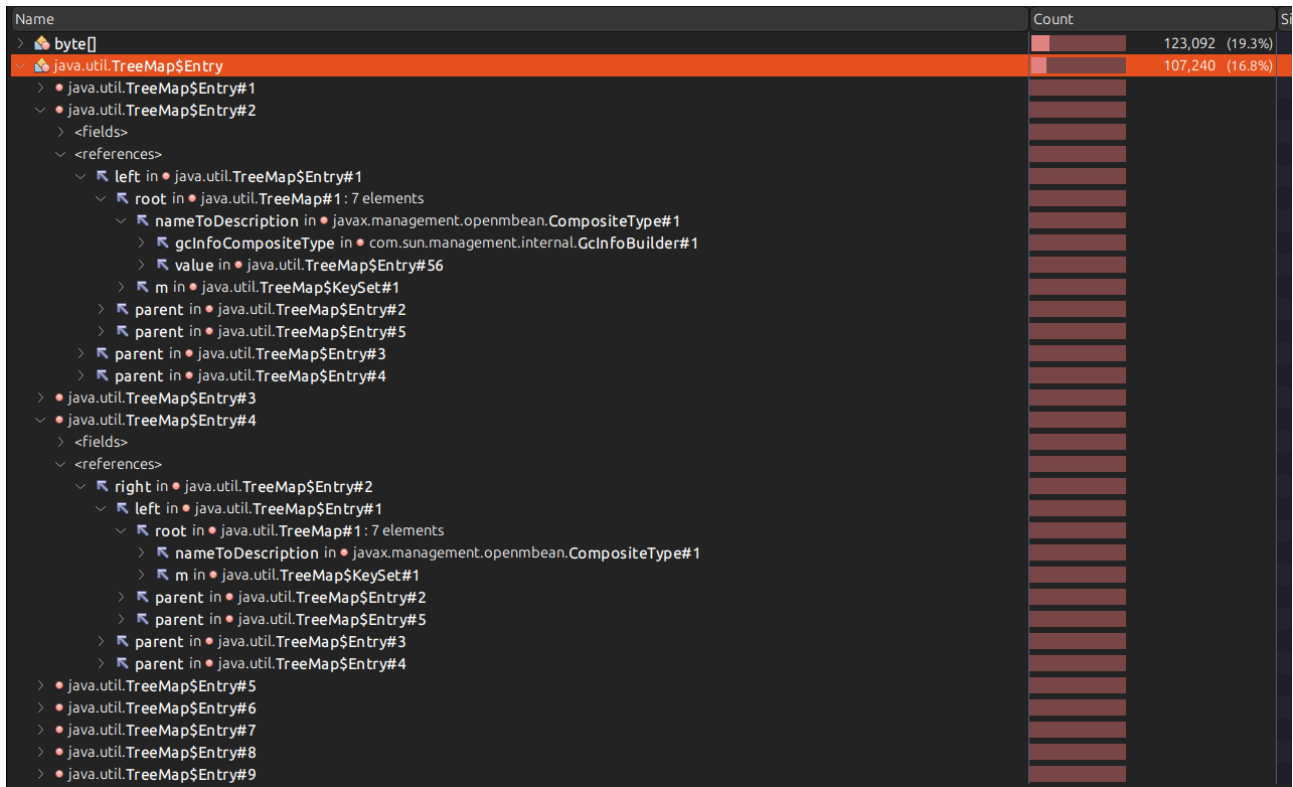


Рис. 11. еще 1 доказательство что я не вру

Если взглянуть немного внимательнее, то видно, что  $n(\text{TreeMap}) \approx n(\text{CompositeDataSupport})$

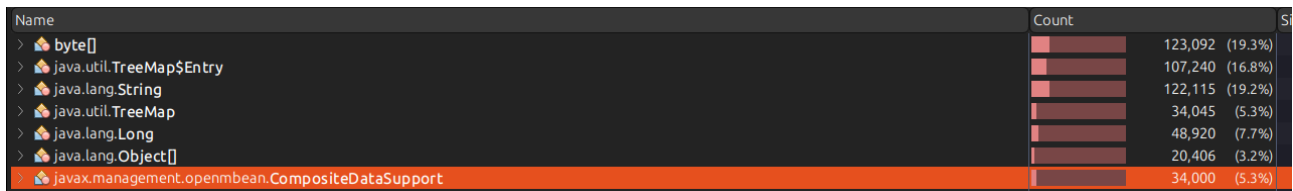


Рис. 12. CompositeDataSupport и TreeMap

Как я вычитал, `CompositeDataSupport` используется, когда JMX-атрибут или метод должен вернуть не примитив, а сложный объект или структуру данных (Map, структуры или кортежа) - а для мониторинга (того же самого **heapdump** это необходимо). Поэтому на это не обращаем внимания.

Посмотрим через **Агрегацию по экземплярам** (ну или как она называется, вот это с точкой), какие объекты занимают больше всего места.



| Name                                                                                     | Size |
|------------------------------------------------------------------------------------------|------|
| java.lang.Object[]#12829: 106,710 items                                                  |      |
| > <items>                                                                                |      |
| > <references>                                                                           |      |
| elementData in java.util.ArrayList#621: 95,222 elements                                  |      |
| static _errorMessagees in class com.meterware.httpunit.javascript.JavaScript: JavaScript |      |

Рис. 15. Object[]#12829.reference

Скорее всего, источник проблемы - это **com.meterware.httpunit.javascript.JavaScript.\_errorMessage**, и скорее всего ЭТОТ СПИСОК НЕ ЧИСТИТСЯ.

```
private static ArrayList _errorMessagees = new ArrayList(); 4 usages

static boolean isThrowExceptionsOnError() { return _throwExceptionsOnError; }

static void setThrowExceptionsOnError( boolean throwExceptionsOnError ) { 1 usage
    _throwExceptionsOnError = throwExceptionsOnError;
}

static void clearErrorMessages() { _errorMessagees.clear(); }

static String[] getErrorMessages() { 1 usage
    return (String[]) _errorMessagees.toArray( new String[ _errorMessagees.size() ] );
}
1
```

Рис. 16. JavaScript.\_errorMessage

Попрыгаем по методу `clearErrorMessages()`:

```
public void clearErrorMessages() { 1 usage
    JavaScript.clearErrorMessages();
}
```

Рис. 17. Первый прыжок на usage

```

/**
 * Clears the accumulated script error messages.
 */
public static void clearScriptErrorMessages() { no usages
    getScriptingEngine().clearErrorMessages();
}

```

Рис. 18. Второй usage

Как видим, метод для очистки накопленных в статическом поле ошибок нигде не используется, как следствие - переполнение памяти.

В качестве решения можно попробовать в Main-классе добавить очистку этого списка после каждого запроса, потому что накапливать его скорее всего нет смысла.

```

43         int number = 1;
44         WebRequest request = new GetMethodWebRequest( urlString: "http://test.me
45         while (true) {
46             WebResponse response = sc.getResponse(request);
47             System.out.println("Count: " + number++ + response);
48             HttpUnitOptions.clearScriptErrorMessages();|
49         //         java.lang.Thread.sleep(200);
50         }
51         //     } catch (InterruptedException ex) {
52         //         Logger.getLogger("global").log(Level.SEVERE, null, ex);
53     } catch (MalformedURLException ex) {
54         Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);

```

Рис. 19. Обновленный Main

Запускаем и видим, что программа спокойно перепрыгивает за 100 000 тиков, что говорит о том, что утечка устранена.

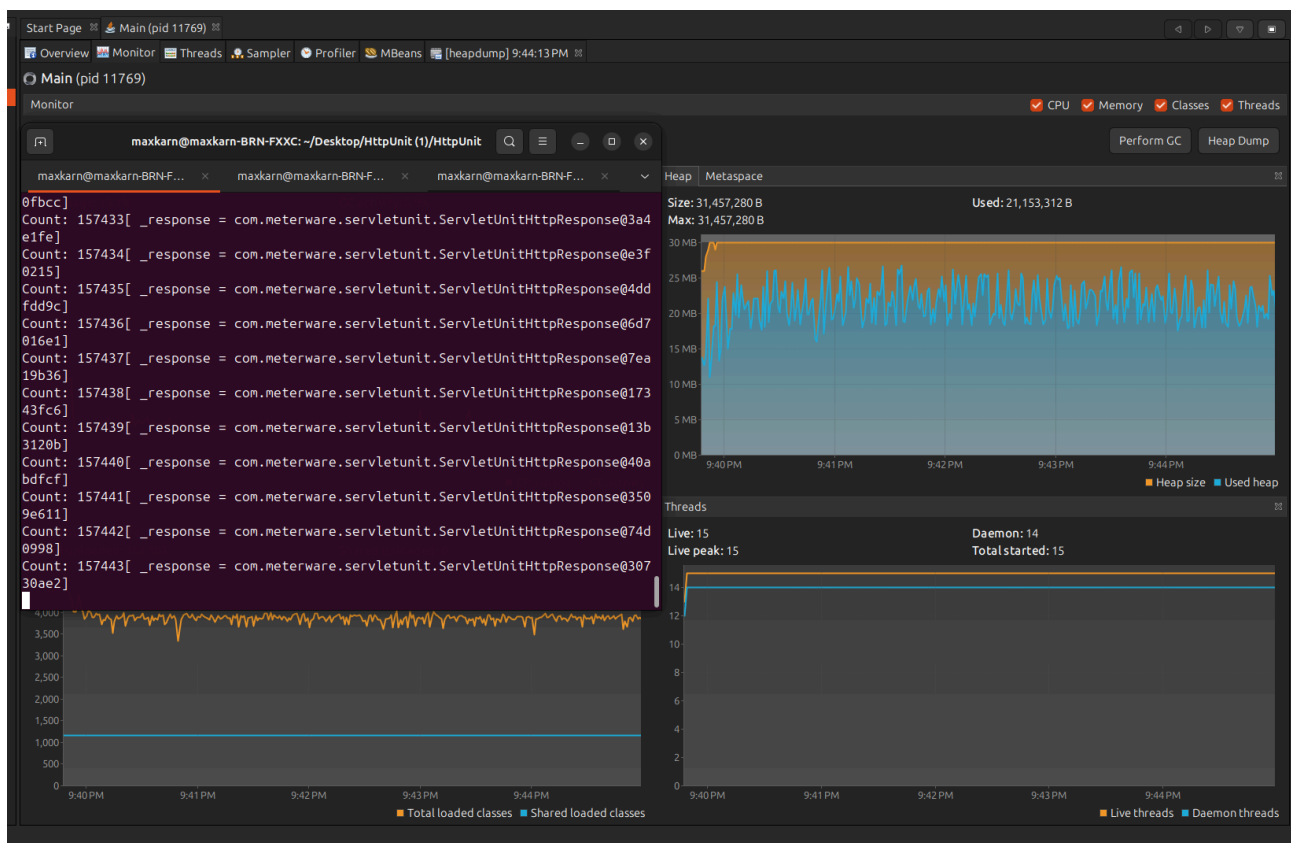


Рис. 20. График хипа после исправления утечки

Как видим, график стабилизировался и у него пропало характерное возрастание вверх. Теперь он просто пологий. Однако мне все равно не очень нравится, что GC практически постоянно активен.

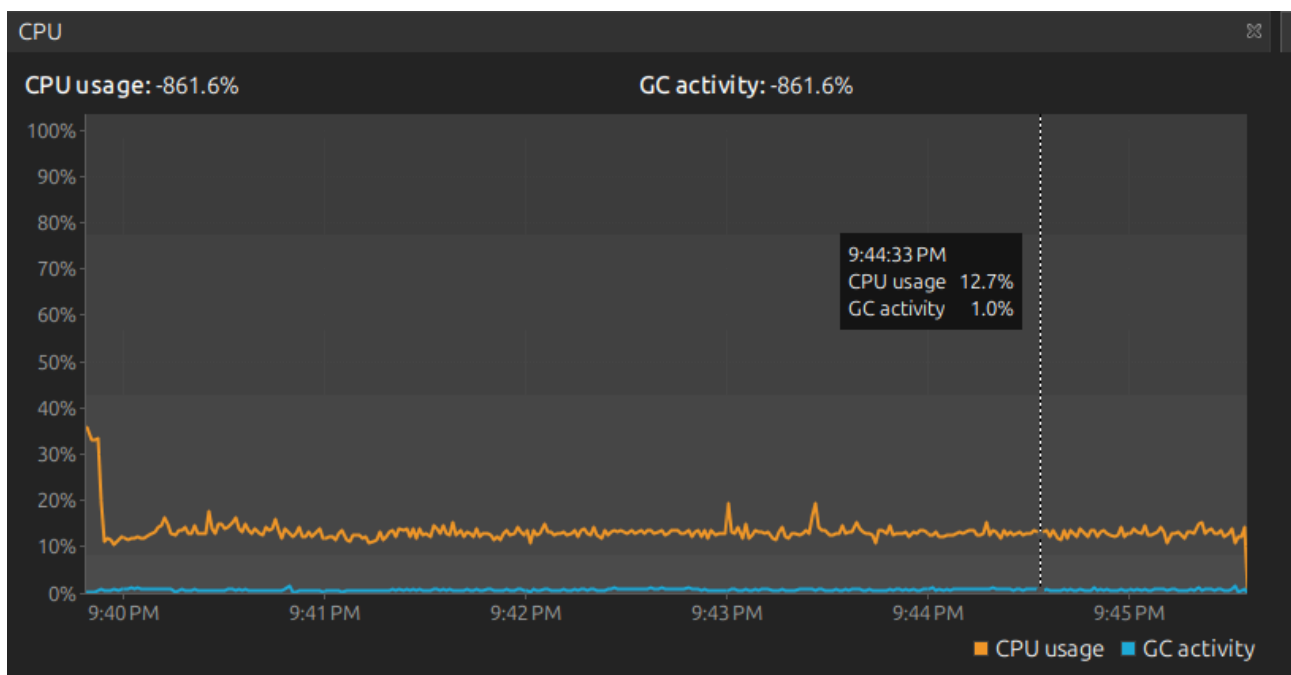


Рис. 21. График активности GC

Думаю, это связано с тем, что я решил убрать `Thread.sleep(200)`, из-за чего запросы стали отправляться чаще, а соответственно и данные на объектном графе стали появляться чаще. Главное, что график не растет вверх!

Взглянем на instance aggregated heapdump:

| Name                                                                                                                    | Size            | Retained (sort to get) |
|-------------------------------------------------------------------------------------------------------------------------|-----------------|------------------------|
| byte[] #619 : 98,473 items                                                                                              | 98,496 B (0.6%) | n/a                    |
| byte[] #615 : 70,821 items                                                                                              | 70,840 B (0.4%) | n/a                    |
| byte[] #15556 : 37,449 items                                                                                            | 37,472 B (0.2%) | n/a                    |
| byte[] #13896 : 37,401 items                                                                                            | 37,424 B (0.2%) | n/a                    |
| java.lang.Object[] #20386 [GC root - JNI global] : 7,937 items                                                          | 31,768 B (0.2%) | n/a                    |
| byte[] #636 : 29,199 items                                                                                              | 29,216 B (0.2%) | n/a                    |
| int[] #881 : 6,712 items                                                                                                | 26,864 B (0.2%) | n/a                    |
| byte[] #608 : 23,831 items                                                                                              | 23,848 B (0.2%) | n/a                    |
| byte[] #601 : 19,438 items                                                                                              | 19,456 B (0.1%) | n/a                    |
| char[] #1 : ...                                                                                                         | 16,400 B (0.1%) | n/a                    |
| char[] #2 : Count: 164145[ _response = com.meterware.servletunit.ServletUnitHttpResponse@60d08f61]...                   | 16,400 B (0.1%) | n/a                    |
| char[] #245 : ...                                                                                                       | 16,400 B (0.1%) | n/a                    |
| int[] #84 : 3,105 items                                                                                                 | 12,440 B (0.1%) | n/a                    |
| char[] #69 : "SSSSSSSSSSSSS&(*,.....02468::<@BDFHJLNPRVWWWWWXVWZ\\     \\^\\ \\ bbbbbb bbbbbb bbbbbb bbbbbb bbbbfhhh... | 12,112 B (0.1%) | n/a                    |
| byte[] #22371 : 12,096 items                                                                                            | 12,112 B (0.1%) | n/a                    |
| byte[] #610 : 10,149 items                                                                                              | 10,168 B (0.1%) | n/a                    |
| byte[] #640 : 9,421 items                                                                                               | 9,440 B (0.1%)  | n/a                    |
| java.lang.Object[] #1902 [GC root - JNI global] : 2,285 items                                                           | 9,160 B (0.1%)  | n/a                    |
| int[] #82 : 2,280 items                                                                                                 | 9,136 B (0.1%)  | n/a                    |
| java.lang.Object[] #11769 : 2,126 items                                                                                 | 8,520 B (0.1%)  | n/a                    |
| byte[] #1539 : 8,192 items                                                                                              | 8,208 B (0.1%)  | n/a                    |
| byte[] #1542 : 8,192 items                                                                                              | 8,208 B (0.1%)  | n/a                    |
| java.util.concurrent.ConcurrentHashMap\$Node[] #37 : 2,048 items                                                        | 8,208 B (0.1%)  | n/a                    |
| int[] #475 : 2,048 items                                                                                                | 8,208 B (0.1%)  | n/a                    |
| java.util.concurrent.ConcurrentHashMap\$Node[] #39 : 2,048 items                                                        | 8,208 B (0.1%)  | n/a                    |
| int[] #773 : 2,048 items                                                                                                | 8,208 B (0.1%)  | n/a                    |
| byte[] #15238 [GC root - Java Frame] : 8,192 items                                                                      | 8,208 B (0.1%)  | n/a                    |
| byte[] #16342 : 8,192 items                                                                                             | 8,208 B (0.1%)  | n/a                    |
| byte[] #16662 : 8,192 items                                                                                             | 8,208 B (0.1%)  | n/a                    |
| byte[] #16692 [GC root - Java Frame] : 8,192 items                                                                      | 8,208 B (0.1%)  | n/a                    |
| java.util.concurrent.ConcurrentHashMap\$Node[] #104 : 2,048 items                                                       | 8,208 B (0.1%)  | n/a                    |
| byte[] #10935 : 7,416 items                                                                                             | 7,432 B (0%)    | n/a                    |
| byte[] #19446 : 6,067 items                                                                                             | 6,088 B (0%)    | n/a                    |

Рис. 22. Самые тяжелые объекты на момент примерно Count: 150 000

Как видим, нашего `Object[]` тут нет.

### 3. Вывод

В ходе выполнения лабораторной работы я познакомился с инструментами мониторинга (JConsole) и профилирования (VisualVM). Попрактиковался в создании Mbean'ов, нахождении утечек памяти, смачно помучался, потому что VisualVM почему-то по кд крашилась, как будто утечка памяти программы распространялась на нее. Но в целом мне зашло. Хотел потрогать IntelliJ'евский профилировщик, но руки не дошли, хватило обычного мониторинга в VisualVM для выявления утечки.